

Pontifícia Universidade Católica de Campinas
Escola Politécnica

ESTUDOS DE BANCO DE DADOS I

Prof Dra Renata Arantes
E-mail: renata.arantes@puc-campinas.edu.br

– TRIGGERS (Oracle)

Automatização de Processos, VIEW e Auditoria de Dados

1. Introdução: o que é uma Trigger

- Uma trigger (gatilho) é um bloco de código PL/SQL executado automaticamente quando ocorre um evento no banco de dados.
- Diferente de funções e procedures, que são chamadas manualmente, triggers são disparadas automaticamente em resposta a comandos INSERT, UPDATE ou DELETE.

2. Onde e quando usar

- Auditoria (registrar logs de alterações)
- Preenchimento automático (datas, usuário, cálculos)
- Regras de negócio (bloquear exclusões, validar dados)
- Evitar uso para processos pesados ou que poderiam ser resolvidos com constraints ou default values.

3. Estrutura geral de uma Trigger

```
CREATE OR REPLACE TRIGGER nome_da_trigger
[BEFORE | AFTER | INSTEAD OF] [INSERT | UPDATE | DELETE]
ON nome_da_tabela
[FOR EACH ROW]
DECLARE
    -- Declarações opcionais
BEGIN
    -- Código a ser executado
    NULL;
END;
```

4. Explicando cada linha

- CREATE OR REPLACE TRIGGER → cria ou substitui a trigger
- BEFORE / AFTER / INSTEAD OF → define o momento de execução
- INSERT / UPDATE / DELETE → define o evento que dispara a trigger
- ON nome_da_tabela → tabela associada
- FOR EACH ROW → atua em cada linha afetada
- :NEW / :OLD → acessa valores novos e antigos do registro

Listagem 1. Sintaxe básica de criação de uma trigger.

```
1 CREATE [OR REPLACE ] TRIGGER trigger_name
```



- CREATE [OR REPLACE] TRIGGER trigger_name - é definido para a criação de uma trigger ou a sobrescrita de uma trigger já existente com o nome informado.

```
6 [REFERENCING OLD AS o NEW AS n]
```

```
7 [FOR EACH ROW]
```

```
8 WHEN (condition)
```

```
9 DECLARE
```

```
10 Declaration-statements
```

```
11 BEGIN
```

```
12 Executable-statements
```

```
13 EXCEPTION Exception-handling-statements
```

```
14 END;
```

Listagem 1. Sintaxe básica de criação de uma trigger.

```
1 CREATE [OR REPLACE ] TRIGGER trigger_name
2   {BEFORE | AFTER | INSTEAD OF }
3   {INSERT [OR] | UPDATE [OR] | DELETE}
4
5   [FOR EACH ROW]
6   WHEN (condition)
7   DECLARE
8   Declaration-statements
9   BEGIN
10  Executable-statements
11  EXCEPTION Exception-handling-
12  END;
```

■ {BEFORE | AFTER | INSTEAD OF} – neste ponto, definimos quando a trigger deverá ser executada. No caso da cláusula INSTEAD OF, esta é utilizada quando uma trigger é criada em uma View.

Quando eu crio uma trigger, eu preciso dizer em que momento quero que ela seja executada:

- 👉 se é antes (BEFORE) — por exemplo, antes de gravar, pra validar ou calcular alguma coisa;
- 👉 se é depois (AFTER) — pra registrar o que aconteceu, tipo gravar um log;
- 👉 ou se é em vez de (INSTEAD OF) — O que eu faria normalmente? Gravar, mas com Instead of, eu digo: não faça isso, faça o que esta dentro da trigger.

O BEFORE e o AFTER trabalham em tabelas reais, mas o INSTEAD OF trabalha em views, que não armazenam dados. Em vez de fazer antes ou depois, ele faz o que tá dentro da trigger.”

Listagem 1. Sintaxe básica de criação de uma trigger.

```
1 CREATE [OR REPLACE ] TRIGGER trigger_name
2   {BEFORE | AFTER | INSTEAD OF }
3   {INSERT [OR] | UPDATE [OR] | DELETE}
4   ON table_name
5   [REFERENCING OLD AS o NEW AS n]
6   [FOR EACH ROW]
7   WHEN (condition)
8   DECLARE
9   Declaration-statements
10  BEGIN
11  Executable-statements
12  EXCEPTION Exception-handling
13  END;
```



- {INSERT [OR] | UPDATE [OR] | DELETE} – neste ponto especificamos uma operação DML.

O back-end não executa a trigger.

Ele apenas envia um comando SQL para o banco, como INSERT, UPDATE ou DELETE.

Quando o banco recebe esse comando, ele verifica se a tabela tem alguma trigger associada.

Se tiver, o Oracle invoca automaticamente a trigger e executa o código dela, de acordo com o que está definido:

se é antes (BEFORE), depois (AFTER) ou no lugar da ação (INSTEAD OF).

Por exemplo:

Se o aluno der um INSERT, o Oracle executa a trigger de inserção.

Se der um UPDATE, ele executa a trigger de atualização.

E se der um DELETE, ele executa a trigger de exclusão.

Tudo isso acontece dentro do banco, automaticamente —

o back-end só dispara o gatilho, e o Oracle faz o resto.

Listagem 1. Sintaxe básica de criação de uma trigger.

```
1 CREATE [OR REPLACE ] TRIGGER trigger_name
2   {BEFORE | AFTER | INSTEAD OF }
3   {INSERT [OR] | UPDATE [OR] | DELETE}
4   [OF col_name]
5   [REFERENCING OLD AS o NEW AS n]
6   [FOR EACH ROW]
7   WHEN (condition)
8   DECLARE
9   Declaration-statements
10  BEGIN
11  Executable-statements
12  EXCEPTION Exception-handling-statements
13  END;
```



- [OF col_name] - especificamos aqui a coluna que será atualizada.

Listagem 1. Sintaxe básica de criação de uma trigger.

```
1 CREATE [OR REPLACE ] TRIGGER trigger_name
2   {BEFORE | AFTER | INSTEAD OF }
3   {INSERT [OR] | UPDATE [OR] | DELETE}
4   [OF col_name]
5   ON table_name
6   [FOR EACH ROW]
7   WHEN (condition)
8   DECLARE
9     Declaration-statements
10  BEGIN
11    Executable-statements
12  EXCEPTION Exception-handling-statements
13  END;
```



- ON table_name - aqui especificamos o nome da tabela que será associada a trigger.

Listagem 1. Sintaxe básica de criação de uma trigger.

```
1 CREATE [OR REPLACE ] TRIGGER trigger_name
2   {BEFORE | AFTER | INSTEAD OF }
3   {INSERT [OR] | UPDATE [OR] | DELETE}
4   [OF col_name]
5   ON table_name
6   [REFERENCING OLD AS o NEW AS n]
7   [FOR EACH ROW]
9   DECLARE
10  Declaration-statements
11  BEGIN
12  Executable-statements
13  EXCEPTION Exception-handling-statements
14  END;
```



- [REFERENCING OLD AS o NEW AS n] – aqui permitimos referências a valores novos e valores antigos para várias declarações do tipo DML.

INSERT = só NEW.
UPDATE = OLD e NEW.
DELETE = só OLD.

Listagem 1. Sintaxe básica de criação de uma trigger.

```
1 CREATE [OR REPLACE ] TRIGGER trigger_name
2   {BEFORE | AFTER | INSTEAD OF }
3   {INSERT [OR] | UPDATE [OR] | DELETE}
4   [OF col_name]
5   ON table_name
6   [REFERENCING OLD AS o NEW AS n]
7   [FOR EACH ROW]
8   WHEN (condition)
9   ■ [FOR EACH ROW] - esta instrução especifica uma trigger no nível de linha.
10  Declaration-statements
11  BEGIN
12  Executable-statements
13  EXCEPTION Exception-handling-statements
14  END;
```

Imagine a tabela notas:

Matrícula	Nota
101	7.0
102	8.0
103	6.5

Sem FOR EACH ROW

→ o banco entende que houve uma atualização geral e a trigger é executada uma única vez, sem saber quais registros foram afetados.

Com FOR EACH ROW

→ o banco executa a trigger 3 vezes, uma pra cada linha alterada, permitindo gravar por exemplo:

Matrícula	Nota Antiga	Nota Nova	Data
101	7.0	8.0	20/10/2025
102	8.0	9.0	20/10/2025
103	6.5	7.5	20/10/2025

Listagem 1. Sintaxe básica de criação de uma trigger.

```
1 CREATE [OR REPLACE ] TRIGGER trigger_name
2   {BEFORE | AFTER | INSTEAD OF }
3   {INSERT [OR] | UPDATE [OR] | DELETE}
4   [OF col_name]
5   ON table_name
6   [REFERENCING OLD AS o NEW AS n]
7   [FOR EACH ROW]
8   WHEN (condition)
9
10  BEGIN
11    Executable-statements
12  EXCEPTION Exception-handling-statements
13
14  END;
```



- WHEN (condição) – neste momento, definimos a condição para que a trigger seja disparada nas linhas.

```
CREATE OR REPLACE TRIGGER tg_auditoria_notas
AFTER UPDATE ON nota
FOR EACH ROW
WHEN (NEW.valor <> OLD.valor)
BEGIN
    INSERT INTO log_auditoria_notas
        (matricula, componente, nota_antiga, nota_nova, data_evento)
    VALUES
        (:OLD.matricula, :NEW.componente, :OLD.valor, :NEW.valor, SYSDATE);

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Erro ao gravar o log de auditoria.');
```

```
END;
```

MATERIAL EXTRA SOBRE SEQUENCE

Oracle SQL Developer : Renata

Arquivo Editar Exibir Navegar Executar Origem Equipe Ferramentas Janela Ajuda



Conexões



Oracle Conexões

Renata

Tabelas (Filtrado)

Views

Índices

Pacotes

Procedures

Funções

Operadores

Filas

Tabelas de Filas

Triggers

Tipos

Sequências

Views Materializadas

Tabelas de Views Materializadas

Relatórios

Todos os Relatórios

Relatórios Definidos pelo Usuário

Relatórios de View Analítica

DBA

Renata.sql

Renata1.sql

Renata2.sql

Renata3.sql

Página de Boas-Vindas

Renata



Planilha

Query Builder

```
create table fornecedor  
( for_cod number (5) CONSTRAINT pk_for_cod PRIMARY KEY,  
  for_nome varchar2 (30) );
```

Saída do Script

Tarefa concluída em 2,096 segundos

Table FORNECEDOR criado.

Oracle SQL Developer : Renata

Arquivo Editar Exibir Navegar Executar Origem Equipe Ferramentas Janela Ajuda



Conexões

Oracle Conexões

- Renata
 - Tabelas (Filtrado)
 - Views
 - Índices
 - Pacotes
 - Procedures
 - Funções
 - Operadores
 - Filas
 - Tabelas de Filas
 - Triggers
 - Tipos
 - Sequências
 - Views Materializadas
 - Classes de Usuário Materializadas

Relatórios

- Todos os Relatórios
- Relatórios Definidos pelo Usuário
- Relatórios de View Analítica

DBA

Conexões

Renata.sql Renata1.sql Renata2.sql Renata3.sql Página de Boas-Vindas Renata

0 segundos

Planilha Query Builder

```
create table fornecedor
( for_cod number (5) CONSTRAINT pk_for_cod PRIMARY KEY,
  for_nome varchar2 (30));

insert into fornecedor values (1, 'Empresal');
```

Saída do Script

Tarefa concluída em 0,079 segundos

Table FORNECEDOR criado.

1 linha inserido.

Oracle SQL Developer : Renata

Arquivo Editar Exibir Navegar Executar Origem Equipe Ferramentas Janela Ajuda



Conexões



Oracle Conexões

- Renata
 - Tabelas (Filtrado)
 - Views
 - Índices
 - Pacotes
 - Procedures
 - Funções
 - Operadores
 - Filas
 - Tabelas de Filas
 - Triggers
 - Tipos
 - Sequências
 - Views Materializadas

Relatórios

- Todos os Relatórios
- Relatórios Definidos pelo Usuário
- Relatórios de View Analítica

DBA



Conexões

Renata.sql Renata1.sql Renata2.sql Renata3.sql Página de Boas-Vindas Renata



Planilha Query Builder

```
create table fornecedor
( for_cod number (5) CONSTRAINT pk_for_cod PRIMARY KEY,
  for_nome varchar2 (30));

insert into fornecedor values (1, 'Empresal');

select * from fornecedor;
```

Saída do Script Resultado da Consulta

Todas as Linhas Extraídas: 1 em 0,176 segundos

FOR_COD	FOR_NOME
1	1 Empresal



Conexões

Oracle Conexões

- Renata
 - Tabelas (Filtrado)
 - Views
 - Índices
 - Pacotes
 - Procedures
 - Funções
 - Operadores
 - Filas
 - Tabelas de Filas
 - Triggers
 - Tipos
 - Sequências
 - Views Materializadas

Relatórios

- Todos os Relatórios
- Relatórios Definidos pelo Usuário
- Relatórios de View Analítica

DBA



```
create table fornecedor
( for_cod number (5) CONSTRAINT pk_for_cod PRIMARY KEY,
  for_nome varchar2 (30));

insert into fornecedor values (1, 'Empresal');

select * from fornecedor;

-- supondo que eu fosse inserir outro
insert into fornecedor values (2, 'Empresa2');
```

FOR_COD	FOR_NOME
1	Empresal
2	Empresa2

Oracle SQL Developer : Renata

Arquivo Editar Exibir Navegar Executar Origem Equipe Ferramentas Janela Ajuda



Conexões



Oracle Conexões

Renata

- Tabelas (Filtrado)
- Views
- Índices
- Pacotes
- Procedures
- Funções
- Operadores
- Filas
- Tabelas de Filas
- Triggers
- Tipos
- Sequências
- Views Materializadas

Relatórios

Todos os Relatórios

- Relatórios Definidos pelo Usuário
- Relatórios de View Analítica

DBA



Conexões

Renata.sql Renata1.sql Renata2.sql Renata3.sql Página de Boas-Vindas Renata



Planilha Query Builder

```
create table fornecedor
( for_cod number (5) CONSTRAINT pk_for_cod PRIMARY KEY,
  for_nome varchar2 (30));

insert into fornecedor values (1, 'Empresal');

select * from fornecedor;

-- supondo que eu fosse inserir outro |
insert into fornecedor values (2, 'Empresa2');

-- Neste caso vai dizer que o registro nao pode ser nulo
insert into fornecedor (for_nome) values (' Empresa3');
```

Saída do Script x Resultado da Consulta x

Tarefa concluída em 0,318 segundos

Table FORNECEDOR criado.

1 linha inserido.

1 linha inserido.

Erro a partir da linha : 13 no comando -

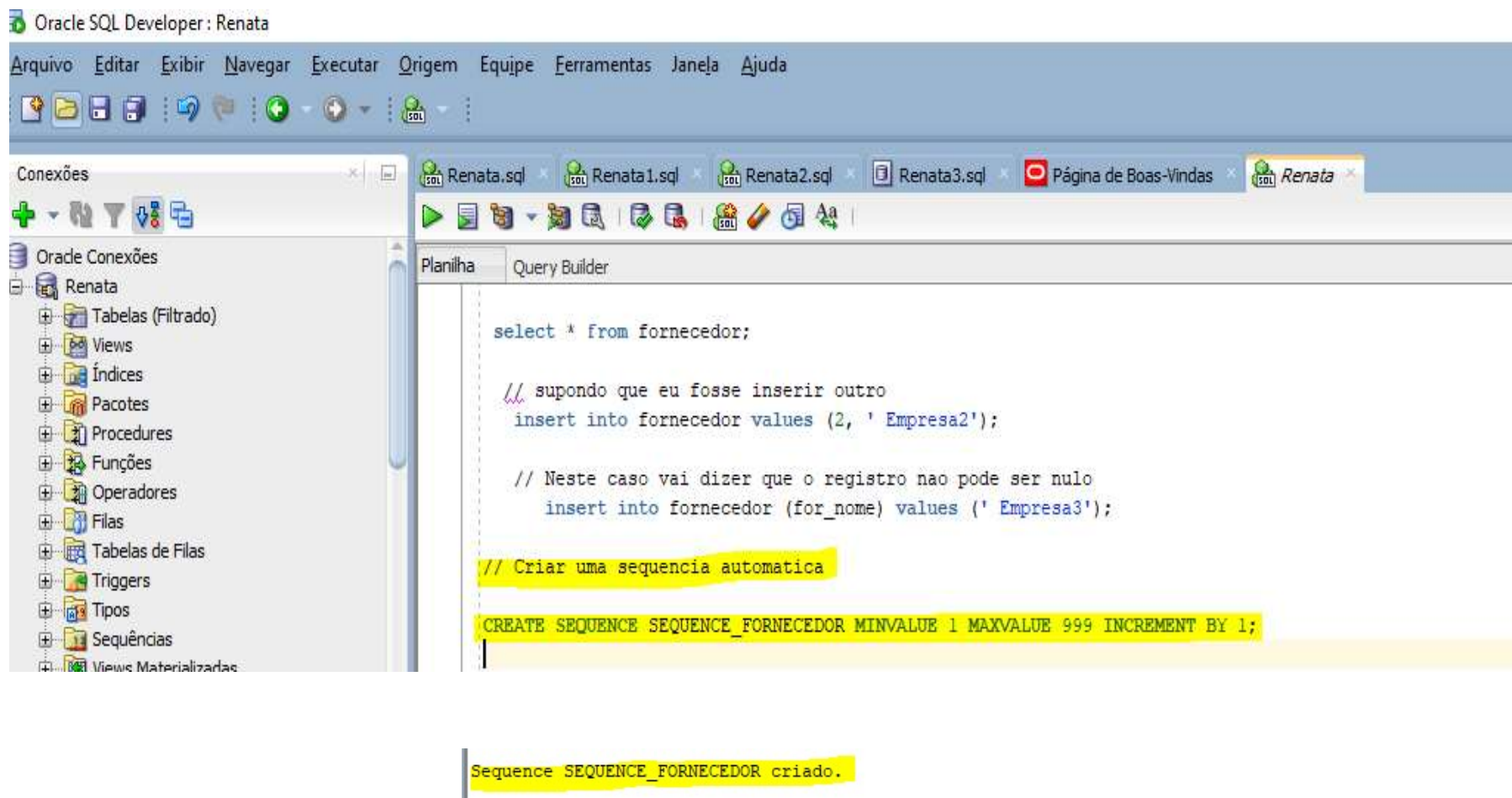
```
insert into fornecedor (for_nome) values (' Empresa3')
```

Relatório de erros -

ORA-01400: não é possível inserir NULL em ("SYSTEM"."FORNECEDOR"."FOR_COD")

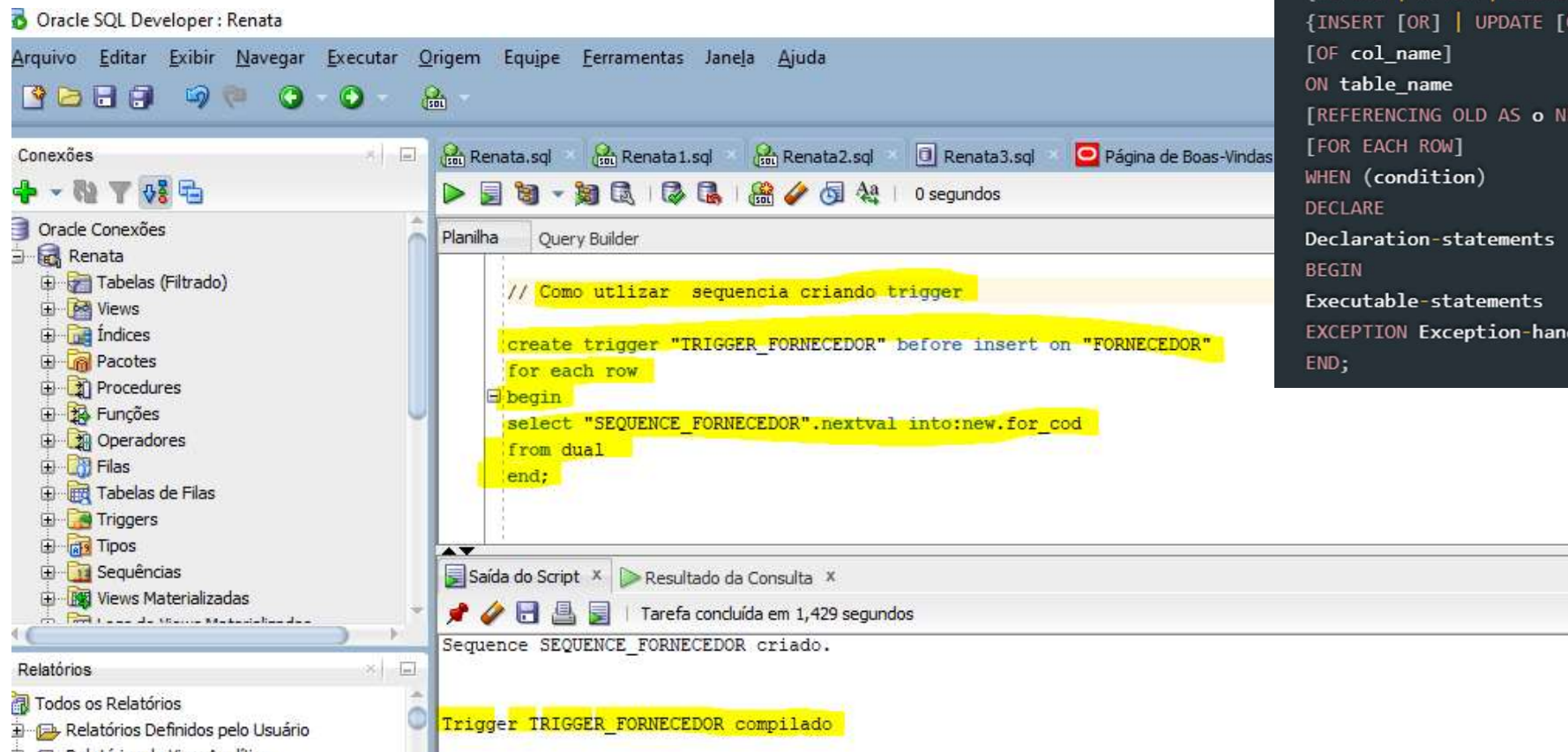
E quando eu não lembrar a sequencia?

- Por isso vamos deixar automático!



Para utilizar essa sequencia?

- Vamos criar uma trigger que vai chamar essa sequencia toda vez que o insert for executado.



The screenshot shows the Oracle SQL Developer interface. The left pane displays the 'Conexões' (Connections) tree with the 'Renata' connection selected. The main editor window shows a SQL script in the 'Query Builder' tab. The script is as follows:

```
// Como utilizar sequencia criando trigger

create trigger "TRIGGER_FORNECEDOR" before insert on "FORNECEDOR"
for each row
begin
select "SEQUENCE_FORNECEDOR".nextval into:new.for_cod
from dual
end;
```

The bottom status bar indicates the script was executed successfully: 'Tarefa concluída em 1,429 segundos' and 'Trigger TRIGGER_FORNECEDOR compilado'.

```
CREATE [OR REPLACE ] TRIGGER trigger_name
{BEFORE | AFTER | INSTEAD OF }
{INSERT [OR] | UPDATE [OR] | DELETE}
[OF col_name]
ON table_name
[REFERENCING OLD AS o NEW AS n]
[FOR EACH ROW]
WHEN (condition)
DECLARE
Declaration-statements
BEGIN
Executable-statements
EXCEPTION Exception-handling-statements
END;
```

Vamos inserir?

- Antes temos que deletar o que tem senão vai dar erro.

```
select * from fornecedor;
```

```
-- supondo que eu fosse inserir outro  
insert into fornecedor values (2, ' Empresa2');
```

FOR_COD	FOR_NOME
1	Empresal
2	Empresa2

```
delete from fornecedor;
```

```
2 linhas excluído.
```

```
// Vamos inserir automatico
```

```
INSERT INTO "FORNECEDOR" (FOR_NOME) VALUES ('Empresal');
```

Saída do Script x

Resultado da Consulta x

Tarefa concluída em 0,052 segundos

Trigger TRIGGER_FORNECEDOR eliminado.

Trigger TRIGGER_FORNECEDOR compilado

1 linha inserido.

```
// Vamos inserir automatico
```

```
INSERT INTO "FORNECEDOR" (FOR_NOME) VALUES ('Empresa1');
```

```
INSERT INTO "FORNECEDOR" (FOR_NOME) VALUES ('Empresa2');
```

```
INSERT INTO "FORNECEDOR" (FOR_NOME) VALUES ('Empresa3');
```

```
select * from fornecedor;
```

Saída do Script x Resultado da Consulta x

SQL | Todas as Linhas Extraídas: 3 em 0,003 segundos

	FOR_COD	FOR_NOME
1	1	Empresa1
2	2	Empresa2
3	3	Empresa3